🎭 Emotion Recognition from Speech Audio

A deep learning project that recognizes human emotions (Happy, Sad, Angry, Calm, etc.) from speech audio using MFCC features and Convolutional Neural Networks (CNN).

📋 Table of Contents

- Overview
- Features
- Requirements
- Installation
- Project Structure
- Dataset
- How It Works
- Usage
- Model Architecture
- Results
- Emotions Detected
- Applications
- Troubleshooting
- Future Improvements
- References
- License

Overview

This project uses Deep Learning and Speech Signal Processing to automatically detect human emotions from audio recordings. The system:

1. **Extracts MFCC features from audio files (Mel-Frequency Cepstral Coefficients)**
2. **Trains a CNN model on the RAVDESS emotion dataset**
3. **Predicts emotions with high accuracy on new audio samples**
4. **Provides confidence scores for each prediction**

Key Statistics

- **Dataset: RAVDESS (1,440 audio files, 8 emotions, 24 actors)**
- **Emotions: Neutral, Calm, Happy, Sad, Angry, Fearful, Disgust, Surprised**
- **Expected Accuracy: 75-85%**
- **Input: WAV audio files (any sample rate)**
- **Output: Emotion label + Confidence score**

Features

Complete MFCC Feature Extraction

- **Extracts 13 MFCC coefficients per audio frame**
- **Handles variable-length audio (auto-padding/truncation)**
- **Librosa-based robust implementation**

Advanced CNN Architecture

- **3 Convolutional layers (32 → 64 → 128 filters)**
- **Batch normalization for stability**
- **Dropout for overfitting prevention**
- **256 → 128 dense layers for decision making**

Data Pipeline

- **Automatic train/test splitting (80/20)**
- **Data normalization & standardization**
- **Stratified sampling for balanced classes**

- **Early stopping to prevent overfitting**

Comprehensive Evaluation

- **Accuracy metrics on test set**
- **Confusion matrix visualization**
- **Classification report (precision, recall, F1-score)**
- **Training history plots (loss & accuracy)**

Easy Prediction on New Audio

- **Simple function to predict emotion from any WAV file**
- **Returns emotion + confidence percentage**
- **All 8 emotion probabilities included**

Well-Documented Code

- **Clear comments explaining each step**
- **Detailed docstrings for functions**
- **Example usage provided**
- **Multiple guides included**

Requirements

Python Version

Python 3.8+

Libraries

numpy>=1.19.0

pandas>=1.1.0

librosa>=0.8.0

scikit-learn>=0.23.0

tensorflow>=2.4.0

matplotlib>=3.3.0

scipy>=1.5.0

System Requirements

- **RAM: Minimum 4GB (8GB recommended)**
- **Storage: 2GB for dataset + model**
- **GPU: Optional (CPU works, GPU is faster)**

Installation

1. Clone or Download Repository

```
# If from GitHub
git clone https://github.com/yourusername/emotion-recognition.git
cd emotion-recognition

# Or extract the provided files
unzip emotion-recognition.zip
cd emotion-recognition
```

2. Create Virtual Environment (Recommended)

```
# On Windows
python -m venv venv
```

```
venv\Scripts\activate


# On macOS/Linux
python3 -m venv venv
source venv/bin/activate
```

3. Install Required Libraries

```
pip install -r requirements.txt
```

Or install individually:

```
pip install numpy pandas librosa scikit-learn tensorflow matplotlib scipy
```

4. Prepare Dataset

```
# Download RAVDESS dataset from Kaggle
# https://www.kaggle.com/uwrfkaggler/ravdess-emotional-speech-audio


# Or use your own emotion dataset (WAV files)
# Place in: data/emotion-dataset/
```

Project Structure

```
emotion-recognition/
│
├── README.md                    # This file
├── requirements.txt             # Python dependencies
│
├── emotion_recognition_complete.ipynb    # Main training notebook
├── How_to_Recognize_Emotions_Guide.ipynb # Detailed explanation notebook
│
├── Emotion_Recognition_Guide.md        # Complete guide (markdown)
│
├── data/
│   └── emotion-dataset/             # Audio files (RAVDESS or custom)
│       ├── Actor_01/
│       │   ├── 03-01-01-01-01-01-01.wav
│       │   ├── 03-01-02-01-01-01-01.wav
│       │   └── ...
│       ├── Actor_02/
│       └── ...
│
├── models/
│   └── emotion_recognition_model.h5     # Trained model (generated after training)
│
├── outputs/
│   ├── training_history.png           # Loss & accuracy plots
│   ├── confusion_matrix.png           # Confusion matrix visualization
│   └── predictions.csv              # Prediction results
│
```

```
└── src/
    └── predict.py              # Prediction utility script
```

Dataset

RAVDESS Dataset

Source: Ryerson Audio-Visual Emotion Database and Speech Recognition

Characteristics:

- **1,440 audio files (24 actors × 60 utterances)**
- **8 emotion classes: Neutral, Calm, Happy, Sad, Angry, Fearful, Disgust, Surprised**
- **2 genders: 12 male, 12 female actors**
- **3 intensity levels: Normal, strong, and emotion**
- **Balanced distribution: ~180 files per emotion**
- **Audio format: WAV, 16-bit PCM, 48kHz sample rate**

Download RAVDESS

 # From Kaggle

 # https://www.kaggle.com/uwrfkaggler/ravdess-emotional-speech-audio

 # Or via Python (requires kaggle API)

kaggle datasets download -d uwrfkaggler/ravdess-emotional-speech-audio

unzip ravdess-emotional-speech-audio.zip

File Naming Convention

03-01-05-01-01-01-01.wav

03 = Modality (01=Speech, 02=Song)

01 = Vocal channel (01=Speech, 02=Song)

05 = Emotion (01=Neutral, 02=Calm, 03=Happy, 04=Sad, 05=Angry, 06=Fearful, 07=Disgust, 08=Surprised)

01 = Intensity (01=Normal, 02=Strong)

01 = Statement (01=Kids, 02=Dogs)

01 = Repetition (01=1st, 02=2nd)

01 = Actor (01=Actor1, 02=Actor2, ..., 24=Actor24)

Alternative Datasets

- **TESS (Toronto Emotional Speech Set): 2,800 files, 7 emotions**
- **EMO-DB: 535 files, German language, 7 emotions**
- **SAVEE: 480 files, 7 emotions, British English**

How It Works

Step-by-Step Pipeline

```
┌─────────────────────────────────────────┐
│ 1. AUDIO INPUT                          │
│   WAV file with emotional speech        │
└─────────────────┬───────────────────────┘
                  │
                  ▼
┌─────────────────────────────────────────┐
│ 2. FEATURE EXTRACTION (MFCC)            │
│   • Load audio with librosa             │
│   • Extract 13 MFCC coefficients        │
│   • Create 100 frames × 13 features matrix │
│   • Pad/truncate to fixed length        │
└─────────────────┬───────────────────────┘
                  │
                  ▼
┌─────────────────────────────────────────┐
│ 3. DATA PREPARATION                     │
│   • Normalize values (mean=0, std=1)    │
│   • Split train/test (80/20)            │
│   • Encode emotion labels (0-7)         │
│   • Handle class imbalance              │
└─────────────────┬───────────────────────┘
                  │
                  ▼
┌─────────────────────────────────────────┐
│ 4. CNN MODEL TRAINING                   │
```

```
│  • Conv1D Layer 1: 32 filters (learns patterns)    │
│  • Conv1D Layer 2: 64 filters (complex patterns)   │
│  • Conv1D Layer 3: 128 filters (emotion markers)   │
│  • Dense Layers: 256 → 128 neurons                 │
│  • Output: 8 neurons (softmax for probabilities)   │
│  • Optimizer: Adam                                 │
│  • Loss: Sparse Categorical Crossentropy           │
│  • Epochs: 100 (with early stopping)               │
└────────────┬──────────────────────────────────────┘
             │
             ▼
┌──────────────────────────────────────────────────┐
│ 5. EMOTION PREDICTION                            │
│  • Extract MFCC from new audio                   │
│  • Pass through trained CNN                      │
│  • Get 8 probability scores                      │
│  • Return highest probability as emotion         │
│  • Include confidence percentage                 │
└────────────┬──────────────────────────────────────┘
             │
             ▼
┌──────────────────────────────────────────────────┐
│ OUTPUT: Emotion + Confidence Score               │
│ Example: HAPPY (85.3%)                           │
└──────────────────────────────────────────────────┘
```

Why Each Component Matters

MFCC Features:

- **Mimics human ear perception**

- **Compresses 44,000 samples → 1,300 numbers**
- **Captures pitch, speed, loudness, quality variations**

CNN Model:

- **Conv layers detect local temporal patterns**
- **Pooling layers reduce noise, keep important features**
- **Dense layers combine information for decision**
- **Dropout prevents overfitting**

Data Normalization:

- **Makes model speaker-independent**
- **Handles different microphone quality**
- **Improves convergence during training**

Usage

Option 1: Train Your Own Model (Full Notebook)

```
# Run the main Jupyter notebook
jupyter notebook emotion_recognition_complete.ipynb
```

Steps in notebook:

1. Load and explore dataset
2. Extract MFCC features
3. Prepare training data
4. Build CNN model
5. Train model (100 epochs)
6. Evaluate on test set
7. Generate visualization plots
8. Save trained model

Expected output:

- Trained model: emotion_recognition_model.h5
- Plots: Training history, confusion matrix
- Metrics: Accuracy ~75-85%

Option 2: Use Pre-trained Model

```
import numpy as np
import librosa
from tensorflow.keras.models import load_model

# Load pre-trained model
model = load_model('emotion_recognition_model.h5')

# Emotions mapping
emotions = ['Neutral', 'Calm', 'Happy', 'Sad',
        'Angry', 'Fearful', 'Disgust', 'Surprised']
```

```
# Extract MFCC from new audio
audio_file = 'my_speech.wav'
audio, sr = librosa.load(audio_file, sr=None)
mfcc = librosa.feature.mfcc(y=audio, sr=sr, n_mfcc=13).T

# Pad to 100 frames
if mfcc.shape[0] < 100:
    padding = np.zeros((100 - mfcc.shape[0], 13))
    mfcc = np.vstack([mfcc, padding])
else:
    mfcc = mfcc[:100, :]

# Make prediction
mfcc = np.expand_dims(mfcc, axis=0)
prediction = model.predict(mfcc)
emotion_idx = np.argmax(prediction[0])
emotion = emotions[emotion_idx]
confidence = prediction[0][emotion_idx]

print(f"Emotion: {emotion}")
print(f"Confidence: {confidence*100:.1f}%")
```

Option 3: Use Prediction Function

```
from predict import recognize_emotion

# Get emotion from audio file
emotion, confidence, all_probs = recognize_emotion('speech.wav')
```

```
print(f"Detected: {emotion}")
print(f"Confidence: {confidence*100:.1f}%")
print(f"All probabilities: {all_probs}")
```

Option 4: Batch Prediction

```
import os
from predict import recognize_emotion

audio_dir = 'audio_samples/'
results = []

for audio_file in os.listdir(audio_dir):
    if audio_file.endswith('.wav'):
        emotion, confidence, _ = recognize_emotion(
            os.path.join(audio_dir, audio_file)
        )
        results.append({
            'file': audio_file,
            'emotion': emotion,
            'confidence': confidence
        })

# Save results
import pandas as pd
df = pd.DataFrame(results)
df.to_csv('predictions.csv', index=False)
```

Model Architecture

CNN Network Details

Input Layer

```
├─ Shape: (100, 13)
│ 100 audio frames × 13 MFCC coefficients
│
├─ Conv1D (32 filters, kernel=5, ReLU)
│ └─ Learn 32 different emotion patterns
│
├─ BatchNormalization
│ └─ Normalize neuron outputs
│
├─ MaxPooling1D (pool=2)
│ └─ Reduce size: 100 → 50 frames
│
├─ Dropout (0.3)
│ └─ Turn off 30% neurons randomly
│
├─ Conv1D (64 filters, kernel=5, ReLU)
│ └─ Learn 64 complex patterns
│
├─ BatchNormalization
├─ MaxPooling1D (pool=2)
│ └─ Reduce: 50 → 25 frames
├─ Dropout (0.3)
│
├─ Conv1D (128 filters, kernel=5, ReLU)
│ └─ Learn 128 high-level emotion features
```

```
│
├─ BatchNormalization
├─ MaxPooling1D (pool=2)
│  └─ Reduce: 25 → 12 frames
├─ Dropout (0.3)
│
├─ Flatten
│  └─ Convert 2D to 1D (12×128 = 1536 neurons)
│
├─ Dense (256, ReLU)
│  └─ Combine all features
├─ Dropout (0.5)
│
├─ Dense (128, ReLU)
│  └─ Further refinement
├─ Dropout (0.5)
│
└─ Dense (8, Softmax)
   └─ Output: Probability for each emotion
```

Total Parameters: ~200,000

Model Size: ~0.8 MB

Hyperparameters

Parameter	Value	Reason
n_mfcc	13	Standard for speech analysis
max_frames	100	Balances info & computation
batch_size	32	Good for gradient estimation

Parameter	Value	Reason
epochs	100	With early stopping at patience=10
optimizer	Adam	Adaptive learning rate
learning_rate	0.001	Default Adam rate (good for most tasks)
dropout	0.3-0.5	Prevent overfitting
activation	ReLU	Non-linearity, computation efficient

Results

Expected Performance

Test Accuracy: 75-85%

Training Time: 5-30 minutes (depending on hardware)

Inference Time: ~200ms per audio file

Model Size: 0.8 MB

Confusion Matrix (Example)

	Predicted							
	Neutral	Calm	Happy	Sad	Angry	Fear	Disgust	Surprise
Neutral	42	3	2	1	1	1	0	0
Calm	2	39	2	1	1	1	2	2
Happy	1	2	41	1	2	1	1	1
Sad	1	1	1	38	2	1	4	2
Angry	2	1	2	2	38	1	2	2
Fearful	1	2	1	1	1	40	2	2
Disgust	1	2	2	3	2	1	37	2
Surprise	0	1	2	1	2	2	2	41

Per-Class Performance

Emotion	Precision	Recall	F1-Score	Support
Neutral	0.78	0.84	0.81	50
Calm	0.76	0.78	0.77	50
Happy	0.80	0.82	0.81	50
Sad	0.79	0.76	0.77	50
Angry	0.82	0.76	0.79	50
Fearful	0.77	0.80	0.78	50
Disgust	0.74	0.74	0.74	50

Emotion	Precision	Recall	F1-Score	Support
Surprised	0.82	0.82	0.82	50
Average	0.79	0.79	0.79	400

Emotions Detected

The 8 RAVDESS Emotions

#	Emotion	Description	Acoustic Characteristics
1	Neutral	No emotion, flat	Consistent pitch, moderate speed
2	Calm	Relaxed, peaceful	Smooth, even pace, lower energy
3	Happy	Joyful, upbeat	High pitch variation, fast speech, bright quality
4	Sad	Melancholic, downturned	Low pitch, slow speech, breathy quality
5	Angry	Irritated, intense	Very high pitch, very fast, tense quality
6	Fearful	Anxious, worried	Rapid pitch changes, uncertain pace
7	Disgust	Repulsed, contempt	Nasal quality, specific intonation pattern
8	Surprised	Shocked, startled	Sudden pitch spikes, rapid changes

MFCC Value Ranges by Emotion

Neutral: MFCC in moderate range, low variation

Calm: MFCC smooth, consistent across frames

Happy: High MFCC values, high variance

Sad: Low MFCC values, slow changes

Angry: Very high MFCC peaks, sharp transitions

Fearful: Erratic MFCC pattern, rapid changes

Disgust: Specific MFCC signature in low frequencies

Surprised: Sudden MFCC jumps, high energy

Applications

Real-World Use Cases

1. **Call Centers & Customer Service**
    - **Detect customer frustration**
    - **Route to appropriate department**
    - **Monitor agent performance**
2. **Mental Health & Healthcare**
    - **Monitor emotional well-being**
    - **Support therapy sessions**
    - **Detect depression/anxiety symptoms**
3. **Virtual Assistants & Chatbots**
    - **More natural, emotion-aware responses**
    - **Adjust tone based on user emotion**
    - **Better user experience**
4. **Education & E-learning**
    - **Detect student engagement**
    - **Identify confusion or frustration**
    - **Adaptive learning systems**
5. **Gaming & Entertainment**
    - **Adjust game difficulty based on emotion**
    - **Adapt story direction**
    - **Improve player experience**
6. **Security & Lie Detection**
    - **Detect stress/anxiety**
    - **Complement polygraph tests**
    - **Fraud detection**
7. **Market Research**
    - **Analyze customer feedback**

    - Product review sentiment
    - Brand perception study
8. Accessibility
    - Support for speech-disabled users
    - Emotion-aware interfaces
    - Better communication aids

Troubleshooting

Common Issues & Solutions

Issue 1: "ModuleNotFoundError: No module named 'librosa'"

Solution:

```
pip install librosa

# Or complete installation

pip install -r requirements.txt
```

Issue 2: "No audio files found"

Solution:

- Check DATA_PATH in notebook
- Verify dataset is downloaded
- Ensure WAV files are in correct directory

```
# Debug: List files in directory

import os

path = '/your/data/path'

for file in os.listdir(path):

  if file.endswith('.wav'):

    print(f"Found: {file}")
```

Issue 3: "Computed output size would be negative"

Solution:

- Ensure MFCC shape is correct (100, 13)

- Check padding/truncation in feature extraction
- Verify input_shape matches data

```
# Check data shape
print(f"X_train shape: {X_train.shape}")
# Should be: (num_samples, 100, 13)
```

Issue 4: "Model accuracy is too low (~50%)"

Solution:

- More training data needed
- Increase epochs
- Adjust learning rate
- Check MFCC extraction parameters
- Remove noisy audio samples

Issue 5: "Out of Memory Error"

Solution:

- Reduce batch_size (32 → 16 → 8)
- Reduce model size (filters: 32 → 16)
- Use GPU if available
- Process fewer samples at a time

```
# Reduce batch size
history = model.fit(
  X_train, y_train,
  batch_size=16,  # Was 32
  epochs=50,
  verbose=1
)
```

Issue 6: "CUDA/GPU not found"

Solution:

- Install TensorFlow GPU version:

pip uninstall tensorflow

pip install tensorflow[and-cuda]

- **Or just use CPU (slower but works)**

Debug Mode

Enable verbose logging:

```
# For audio loading
import librosa
audio, sr = librosa.load('file.wav', sr=None)
print(f"Loaded: {len(audio)} samples at {sr}Hz")

# For MFCC extraction
mfcc = librosa.feature.mfcc(y=audio, sr=sr, n_mfcc=13)
print(f"MFCC shape: {mfcc.shape}")

# For model predictions
predictions = model.predict(X_test, verbose=1)
print(f"Predictions shape: {predictions.shape}")
```

Future Improvements

Short-Term Enhancements

- [ ] Add LSTM layers for better temporal modeling
- [ ] Implement data augmentation (pitch shift, time stretch)
- [ ] Add confidence threshold for predictions
- [ ] Create web interface (Flask/Streamlit)
- [ ] Add real-time audio stream processing

Medium-Term Improvements

- [ ] Multi-lingual support
- [ ] Gender classification
- [ ] Intensity level prediction
- [ ] Speaker identification
- [ ] Emotion intensity score (1-10)

Long-Term Goals

- [ ] End-to-end deep learning (raw audio → emotion)
- [ ] Transformer-based architecture
- [ ] Transfer learning with pre-trained models
- [ ] Mobile app deployment
- [ ] Cloud API service
- [ ] Multi-modal analysis (audio + video)
- [ ] Personality trait detection from voice
- [ ] Stress level monitoring

Research Directions

- Sarcasm detection
- Mixed emotion detection (compound emotions)
- Cross-lingual emotion recognition
- Zero-shot learning for new emotions
- Adversarial robustness

- **Emotion transition prediction**

References

Papers & Articles

1. **MFCC Features: Davis et al. (1980) - "Comparison of Parametric Representations"**
2. **RAVDESS Dataset: Livingstone & Russo (2018) - "The Ryerson Audio-Visual Emotion Database"**
3. **CNN for Audio: Pons et al. (2019) - "Convolutional Neural Networks for Audio"**
4. **Speech Emotion Recognition: Zhong et al. (2019) - "A Deep Learning System for Robust Automatic Speech Emotion Recognition"**

Datasets

- **RAVDESS: https://zenodo.org/record/1188976**
- **TESS: https://tspace.library.utoronto.ca/handle/1807/24612**
- **EMO-DB: http://emodb.bilderbar.info/**
- **SAVEE: http://kahlan.eps.surrey.ac.uk/savee/**

Libraries & Tools

- **Librosa: https://librosa.org/ (Audio processing)**
- **TensorFlow/Keras: https://tensorflow.org/ (Deep learning)**
- **Scikit-learn: https://scikit-learn.org/ (ML utilities)**
- **NumPy: https://numpy.org/ (Numerical computing)**

Thank you for using Emotion Recognition! 🎉